A Hybrid LSTM-Transformer Model for Power Load Forecasting

Vasileios Pentsos, Spyros Tragoudas, *Senior Member, IEEE*, Jason Wibbenmeyer, and Nasser Khdeer

***Abstract*—This paper introduces a novel optimized hybrid model combining Long Short-Term Memory (LSTM) and Transformer deep learning architectures designed for power load forecasting. It leverages the strengths of both LSTM and Transformer models, ensuring more accurate and reliable forecasts of power consumption while considering geographic factors, user behavioral factors, and time constraints for the training time. The model is modified to forecast the total power load for consecutive future time instances rather than the next time instance. We have tested the models using residential power consumption data, and the findings reveal that the optimized hybrid model consistently outperforms existing methods.**

***Index Terms*—Artificial intelligence, LSTM, Transformer, hybrid architecture, power load forecasting, optimization, smart grid.**

I. Introduction

ACCURATE power load forecasting using historical data is crucial for utilities and electricity providers because it supports efficient power generation and minimizes service interruptions. In residential smart grids, advanced data analysis is employed to predict consumption patterns and automate monitoring, thereby enhancing demand forecasting. Such accuracy not only facilitates better energy management and tailored billing but also optimizes supplier profits and resource allocation within the distribution network, ensuring both sustainability and profitability. Forecasting power demand at the individual user level, however, remains complex due to its reliance on the specific behaviors and habits of residents. Accurate predictions become feasible in scenarios where users maintain a consistent daily routine involving activities like heating, laundry, or dryer usage. In contrast, the absence of such consistent patterns poses significant challenges to accurately forecasting residential power demand.

Received 4 January 2024; revised 15 June 2024 and 5 October 2024; accepted 28 December 2024. Date of publication 13 February 2025; date of current version 23 April 2025. The work of Vasileios Pentsos and Spyros Tragoudas was supported in part by NSF under Award 2231623, and in part by Intelligent, Distributed, Embedded, Applications, and Systems (IDEAS) NSF Industry-University Cooperative Research Centers (IUCRC). Paper no. TSG-00025-2024. *(Corresponding author: Vasileios Pentsos.)*

Vasileios Pentsos and Spyros Tragoudas are with the School of Electrical, Computer, and Biomedical Engineering, Southern Illinois University at Carbondale, Carbondale, IL 62901 USA (e-mail: vasileios.pentsos@siu.edu).

Jason Wibbenmeyer is with the Corporate Planning Department, Ameren, St. Louis, MO, USA.

Nasser Khdeer is with the Critical Infrastructure Department, Ameren, St. Louis, MO, USA.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TSG.2025.3535407.

Digital Object Identifier 10.1109/TSG.2025.3535407

Several approaches have been employed for power load forecasting, encompassing traditional statistical methods like ARIMA [1] and more contemporary machine learning (ML) and deep learning (DL) techniques. DL techniques leverage neural networks to capture intricate relationships among various variables. Notably, various adaptations of Long Short-Term Memory (LSTM) [2] architectures and combinations of LSTM with Convolutional Neural Networks (CNN) have found widespread application in the literature for similar tasks [3], [4], [5], [6], [7], [8], [9], [10], [11], [12].

Kong et al. [7] utilized an LSTM model for residential forecasting with manually selected load categories as inputs. In a separate investigation [8], an LSTM model forecasted total residential load demand, incorporating variables such as historical data and specific time/day information. The works in [8], [13] introduced algorithms for predicting total power consumption based on per-house data. In our recent study [14] we optimized specific parameters typical in deep learning models, showcasing their significant capability in delivering accurate power demand predictions across various scenarios that encompass individual residential units and networked residential configurations. Bi-directional Long Short-Term Memory (BiLSTM) networks are a variation of the standard LSTM architecture introduced by Huang et al. [15]. In their work [16], Guo et al. proposed a load forecasting approach featuring BiLSTM combined with meteorological data to forecast multiple load categories.

Lin et al. in [13] proposed a short-term forecasting model utilizing graph neural networks (GNN) to account for spatial correlations in power demand among neighboring houses. The model constructed a graph for a specific household based on actual residential distances and trained GNN. However, the feasibility of real-time applications is questioned due to the differentiation of inter-house relations over time. The training process is reported to be time-consuming, as highlighted in [13], posing a challenge for timely calculations.

Transformers represent deep feed-forward artificial neural networks equipped with a self-attention mechanism [17]. These models were originally proposed for natural language processing tasks [18], [19], [20], [21], but have exhibited remarkable achievements in diverse domains and various other fields of application. In power load forecasting, Transformer models have recently gained attention for sequence-to-sequence prediction, being employed both independently and in conjunction with existing architectures [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33]. Transformer models typically require longer training times than LSTM

1949-3053 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.

models for power load forecasting due to their complexity and larger parameter sets. While some models have used the standard encoder-decoder Transformer architecture [22], [25], [26], [34], others have focused solely on either the encoder [24] or decoder [35] module of the Transformer model. Also, Wang et al. in [32] forecasted multiple loads and used a single encoder with multiple decoders, one decoder per forecasted load.

Combinations of LSTM and Transformer models have been utilized by Yan et al., in [36], with two attention mechanisms that feed a BiLSTM for short-term forecasting energy loads. The authors in [37] explored the forecasting capability of hybrid structures when LSTM feed Transformer. They investigated various forecasting problems, among which load forecasting. For each load forecasting their focus is to identify optimal time series correlations in order to construct a multivariate LSTM that feeds Transformer to forecast the load. The works in [36] and [37] motivated us to propose a similar approach tailored to our specific requirements.

In [38] a scheme of two LSTMs with Transformer blocks in the middle of them was introduced for the task of image captioning, and [39] proposed a hybrid model that combines Convolutional Long Short-Term Memory (ConvLSTM) and Transformer networks to forecast the remaining usable life (RUL) of mechanical bearings. The objectives of [38] and [39] are not relevant to load forecasting.

This work introduces a novel hybrid architecture that leverages the power of LSTM and Transformer models to effectively handle multiple input variables for forecasting the total power demand for multiple units (houses), each consisting of multiple loads (appliances). We concentrated on investigating key factors pertinent to load time series analysis. We conducted a thorough examination of the correlations between appliances within individual households (intra-house) and between appliances across different households (inter-house), revealing the importance of maintaining appliance groupings within the same household. Furthermore, our study extended to analyze the influence of geographic factors by generating clusters of households, with each cluster comprising homes from the same U.S. state. We optimized the hybrid LSTM-Transformers model to capture temporal patterns and improve the accuracy of predictions for the total power consumption. An algorithm is also presented to determine the minimum number of hybrid models needed to forecast the total power of a cluster of houses, subject to user-defined time constraints for the training time. Forecasting can be improved by considering loads from households outside the cluster, as this may result in better-correlated load time series. Such optimizations are investigated in [37], but they are not scalable for the problem addressed in the present work.

Furthermore, the proposed hybrid model is modified to forecast the total power load for consecutive future time instances rather than the next time instance. Our experimental evaluation on real-world data demonstrates significant improvements in accuracy compared to existing methods. Environmental factors, such as temperature, and atmospheric pressure could potentially enhance predictive capabilities, but they were not accessible in the present study.

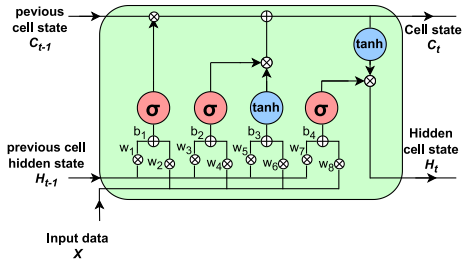


Fig. 1. The LSTM cell i in the LSTM architecture.

The remainder of the paper is organized as follows. Section II details the existing deep learning models that constitute the building blocks of our proposed architecture. The proposed methodology is detailed in Section III. In Section IV we present the experimental results which show superiority in load forecasting when compared to existing approaches, including our recent work in [14] that uses LSTMs. Finally, Section V concludes.

II. Preliminaries

A. The LSTM Model

A Recurrent Neural Network (RNN) is a type of Artificial Neural Network designed for processing sequential data. Among RNNs, Long-Short-Term-Memory (LSTM) models are a specialized variant known for their ability to maintain information over extended periods and capture long-term dependencies in data. The fundamental unit of an LSTM is called the cell, visualized in Figure 1. At any given time step, the cell receives inputs, which include the previous cell state (the long-term memory), the previous cell hidden state (the short-term memory), and the current input data. It produces two outputs: the cell state (updated long-term memory) and the cell hidden state (updated short-term memory).

The input data is a multivariate array denoted as $\mathbf{X} = X_{t-j}^{i}$, with dimensions $N \cdot M$, where $1 \leq i \leq M$ and $1 \leq j \leq N$. Here, we have M time sequences of loads. Each sequence comprises N time instances, which represent the number of preceding time steps the cell considers when making predictions. We refer to this as the **look back** parameter, denoted by N. The output of the LSTM module is T-sized embedding vectors, denoted as $\mathbf{Y}$.

The **look forward**, or look ahead parameter, denoted by F, indicates the number of future time steps the network predicts. It can be either 1, when the cell predicts the total load of the next time sequence, or greater than 1 when the cell provides predictions for multiple future time instances. These parameters are set during the definition of the LSTM and determine how far back in time the network retains information and how far into the future it extends its predictions. Within the LSTM cell architecture, the input data denoted as X and the previous cell's hidden state H_{t-1} undergo four distinct operations. In each operation, the pair of these entities undergoes multiplication with their corresponding weight matrices w_i, where $1 \leq i \leq 8$, followed by addition with a bias term b_j, where $1 \leq j \leq 4$. The resulting sum is then subjected to an activation function, which can take the form of either a Sigmoid (denoted by σ) or a hyperbolic tangent function

Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.

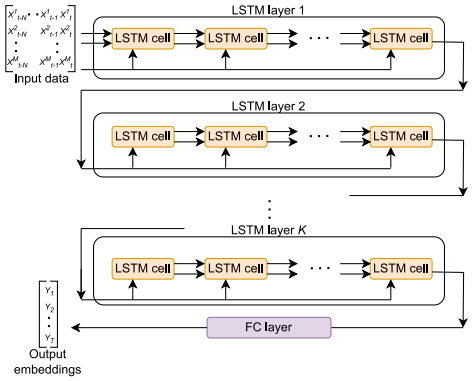


Fig. 2. The LSTM module utilized in the proposed architecture.

(denoted by *tanh*), as depicted in Figure 1. The outputs of the LSTM cell comprise the cell state C_t and the hidden cell state H_t.

In the architecture of the LSTM module in our work, there are multiple LSTM layers, and each layer contains a sequence of LSTM cells. These cells are connected in a chain, with the output of one cell feeding into the next, as can be seen in Figure 2. As data passes through these layers, each subsequent layer receives input from the preceding one. Importantly, the operations within each cell of a particular layer share identical weights and biases. The output of the final LSTM layer is directed to a fully connected layer (FCL), where the output of each activation function of the previous layer feeds all the activation functions in the FCL. The FCL acts as a mapping from the LSTM's output to the desired output dimension T, generating embedding vectors $\mathbf{Y}$. This design ensures consistent feature extraction across the time series data. The final output of the LSTM module is a vector representation of the input data. LSTMs are designed to manage long-term dependencies, however, their effectiveness diminishes with increasing sequence length due to issues such as the vanishing gradient problem [40], [41]. This makes it challenging for LSTMs to capture dependencies on lengthy sequences. For example, our experiments in [14] revealed that extending the look-back window beyond sequences of 90 instances resulted in non-converging loss function values, rendering training ineffective.

B. The Transformer Model

The Transformer model consists of an encoder-decoder structure. The encoder and decoder both incorporate residual connections that feed into Add & Norm blocks, which are essential for stabilizing and improving the training process. The encoder's primary function is to convert the input sequence into a set of attention-based representations, while the decoder uses these representations, along with its own self-attention mechanisms, to generate the output sequence.

The Transformer architecture is characterized by its self-attention mechanism, a pivotal feature that enables it to capture dependencies between various elements in a sequence without relying on recurrent connections. This mechanism empowers the model to selectively focus on specific segments of the input sequence, an ability crucial for power load forecasting. In particular, the self-attention mechanism is realized through the Scaled Dot-Product Attention, as defined in [17], and can be formally described as a function that maps a Query and a set of Key-Value pairs to an output. Queries, Keys, Values, and the resulting output are all vector representations. Let d_k represent the dimension shared by both Query and Query vectors and d_v be the dimension of the Value vector. The attention mechanism hinges on three primary matrices: the Query matrix ($\mathbf{Q}$), the Key matrix ($\mathbf{K}$), and the Value matrix ($\mathbf{V}$). The Query vector is multiplied by the Key vectors of all elements in the sequence to measure the similarity between the Query and each Key. The result of the dot product is divided by the square root of the dimension of the Key vector to prevent the dot products from becoming excessively large, which could push the Softmax function into regions with extremely small gradients. The scaled dot products are passed through a Softmax function to obtain the attention weights. This converts the scores into probabilities, ensuring they sum to one. The Softmax function highlights which elements in the sequence are most relevant to the current element being processed.

The attention mechanism can be expressed as a matrix calculation depicted in Equation (1), as defined in [17].

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

The attention mechanism assesses the relevance of keys to queries and utilizes this relevance to weigh the associated values. The weighted values are then aggregated to derive the output of the attention layer, which constitutes a vector capturing pertinent information from the input sequence concerning the current prediction. In the context of power load prediction, the encoder takes the output of the LSTM component, denoted as vector $\mathbf{Y}$ in Section II-A, as its input. It's important to note that positional encoding is initially applied since the Transformer lacks inherent knowledge of the order or position of elements in the input. Positional encodings, based on sine and cosine functions, are added to the input embeddings to impart positional information to the model. In the present work, we followed the implementation of [17] for positional encoding. The dimension of these positional encodings aligns with the dimension of the embeddings, denoted as T. The formula for computing positional encodings for each position (pos) and dimension (i) is as follows:

$$\text{position_enc}(pos, i) = \begin{cases} \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{pos}{10000^{\frac{2(i-1)}{d_{\text{model}}}}}\right) & \text{if } i \text{ is odd} \end{cases} \tag{2}$$

Here, pos signifies the position in the sequence, i denotes the dimension, and d_{model} is the model's dimensionality.

The encoder architecture, illustrated in Figure 3, is composed of multiple self-attention layers working in parallel to form an intricate multi-head attention mechanism. Following the Multi-Head Attention layer, the encoder incorporates a Feed-Forward, fully connected network, with residual connections, both feeding a normalization layer (Add and Norm). Starting with the input sequence ($\mathbf{Y}$), the matrices ($\mathbf{Q}$, $\mathbf{K}$,

Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.

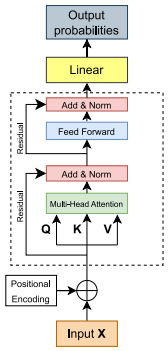


Fig. 3. The utilized Transformer encoder.

and **V**) are generated and successively processed through both the Multi-Head Attention and the Feed-Forward Network. The outcome is the prediction of the power load at the next time instance. For classification tasks, this prediction would typically involve a Softmax activation function to map input data into discrete categories. However, in our power load forecasting task, which is a regression, the output of the Linear layer provides direct continuous value predictions without the use of Softmax.

The decoder generates the output sequence one element at a time, attending to the encoder's output and the previously generated elements in the output sequence. Each layer in the decoder consists of three basic building blocks. The masked Multi-Head Self-Attention Mechanism is similar to the encoder's Multi-Head Attention but with an additional masking step to ensure that each position can only attend to earlier positions in the output sequence. The Encoder-Decoder Attention consists of a second Multi-Head Attention mechanism that allows the decoder to attend to the encoder's output. Finally, the Feed-Forward Network, which is similar to the encoder's FFN, is applied to each position separately. The decoder also employs residual connections and layer normalization in each sub-layer. Due to its more complex structure, training a Transformer model is on average more time-consuming compared to training an LSTM model.

III. The Proposed Method

In this section, we provide a detailed exposition of the methodology underpinning our proposed hybrid LSTM-Transformer model for power load forecasting. Furthermore, we delve into the developed techniques to optimize our hybrid model. We also modify the hybrid model to forecast the loads for multiple consecutive time instances.

A. LSTM-Transformer Model for Load Forecasting

Our proposed hybrid model represents a fusion of the Long Short-Term Memory (LSTM) and the Transformer models. This novel architecture is tailored to efficiently capture and leverage the temporal dependencies within multivariate time series data for enhanced power load forecasting. The core of our hybrid model is a sequential structure that adeptly processes the input multivariate time series data and comprises two main components.

At the forefront of the architecture lies the LSTM component. This component handles intricate temporal relationships present in the input data **X**. Unlike traditional standalone LSTM models that typically perform regression tasks, the LSTM module in this context primarily focuses on generating meaningful embeddings to represent the input data. These embeddings capture essential features and patterns inherent in the multivariate load time series data. The LSTM module achieves this by undergoing a sequence of operations, iterating over the input data to produce embedding vectors of dimension T. The LSTM is designed with K layers, each comprising L LSTM cells, feeding a fully connected layer that generates the T-sized embeddings. The values of T, K, and L are empirically optimized through a systematic trial and error process to achieve the best performance metrics on our forecasting tasks.

LSTMs exhibit a formidable ability to comprehend and learn intricate temporal dynamics from sequential data. By incorporating an LSTM as the initial component, we allow the model to effectively capture essential sequential features before passing the processed information to the Transformer component. Given that our primary goal is to predict the total load consumption for the subsequent time frame, a regression task that aligns with the encoder's capabilities, the encoder-only approach emerges as a reasonable choice, which additionally allows for minimizing model complexity and training time. Furthermore, by bypassing the decoder, we mitigate computational overhead, which is particularly pertinent for lengthy sequences.

The second component of the proposed architecture is the Transformer encoder, a component responsible for generating predictions of load consumption from the vector embeddings obtained in the first module (LSTM). The encoder represents the fundamental aspect of the original Transformer architecture [17], excluding the decoder module. Initially, the encoder augments the embeddings with positional encodings, essential for preserving the sequential order of the input data. Since the input contains already numeric values, an input embedding layer, that is part of the originally proposed mechanism, is not beneficial for the examined task.

These enhanced embeddings are then channeled into the main encoder body, composed of P identical stacked layers.

Within each layer, the encoder employs a multi-head attention mechanism, facilitating parallel self-attention operations. Specifically, we utilize R attention heads per attention layer to effectively capture various aspects of the input data. The keys, values, and queries for these attention heads are derived from the previous layer's output, ensuring a continuous flow of contextual information. Furthermore, each layer incorporates a fully connected feed-forward network, applied independently to each position in the input sequence, promoting richer feature representation. The proposed architecture called LT_E for the remainder of this paper, can be depicted in Figure 4, and is designed to predict the power load for the next time instance.

The MAE metric is defined as:

$$MAE = \frac{1}{n}\sum_{j=1}^{n}\left|\hat{y}_j - y_j\right| \tag{3}$$

Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.

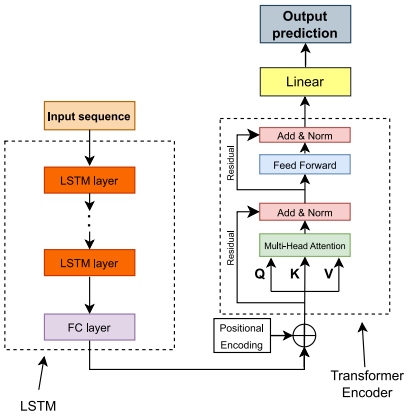


Fig. 4. The proposed hybrid LT_E model.

where y_j corresponds to the real value at the time instance j, $\hat{y}_j$ is the corresponding predicted value for the same time instance, and n is the number of the total time instances.

The Mean Absolute Error (MAE), as defined in Eq.3, measures the average of the absolute discrepancies between each model's forecasted total load and the actual load for each house cluster. However, the direct application of raw MAE as a performance indicator may lead to misleading conclusions, particularly in scenarios exhibiting wide variations in total consumption among different clusters. Notably, clusters with higher consumption levels tend to show increased MAE values, which could misleadingly suggest reduced model accuracy. To address this misinterpretation, we introduced a normalization process for the MAE, adjusting it in proportion to the total consumption of each cluster. This adaptation, termed the *Percentage MAE* (PMAE), is computed by dividing the MAE of each cluster's predictions by its total consumption, thus normalizing the error metric across clusters with diverse consumption levels. PMAE is formally defined as:

$$\text{PMAE} = \frac{\text{MAE}}{\text{Total Consumption}} \tag{4}$$

where MAE is calculated from Eq. 3 for a cluster of houses, and Total Consumption is the sum of all actual consumption values for the cluster over the observed period.

Our analysis of residential power consumption data revealed a sensitivity of LT_E performance to the look-back parameter N, which can be attributed to the LSTM component of the architecture, as reported in [22]. Therefore, we adopt an empirical technique to determine the most effective look-back window for analyzing large housing datasets. This method is fundamentally statistical, aiming to determine a representative look-back parameter that reflects the entirety of the dataset without requiring a predefined collection of houses. The approach can significantly reduce the training time, a critical advantage when forecasting the total load of diverse housing collections. The process starts by establishing a range of possible look-back values, setting an upper limit, and selecting random house collections of varying sizes from the dataset. For each collection, we train an LT model to predict its total load, by varying the look-back window for each model while keeping other model parameters constant.

Algorithm 1 Time_Constraint Unit Clustering

Inputs: N (optimal look-back window) from statistical method,
Q (the number of households), M (the total appliances), T_{th} (training time threshold)

$n \leftarrow 1$ (initializing number of clusters)
while $n \leq Q$ **do**
  Randomly partition Q into n clusters of $\frac{Q}{n}$ units.
  for each cluster i in n **do**
    Train an $LT_{E,i}$ model using the appliances in i.
    Store the training time t_n of the model.
  Aggregate the outputs of all n models $LT_{E,i}$ in Q.
  if $t_n \leq T_{th}$ **then**
    Evaluate the aggregated output using Eq.(3).
  $n \leftarrow n + 1$

Output: Optimal cluster size n subject to T_{th}.

Consider a power substation where a controller manages a network of Q units, equipped with M total load series. We chose to cluster load time series based on units, i.e., keeping the appliances of each house together within the input time series, because they are strongly correlated series. In contrast, we observed that load time series in different houses are not strongly correlated. There are two extreme cases on how to cluster the appliances of the substation: one when all the load time series of the units are handled by a single hybrid model that outputs the total power load forecasting, and another when the appliances of each house are handled by individual hybrid models, and aggregate their predictions to output the total power load forecasting of the substation. As will be shown in Section IV, the strategy that yields the lowest error in the total power prediction of a cluster is the scenario where all the load series are handled by a single model.

The relationship between the number of input variables and the corresponding training time of hybrid LT_E models is linear, as will also be shown in Section IV. Thus, if the number of input variables of a cluster is large, the training time may be prohibitively long for practical real-world applications. Furthermore, such applications may require periodic retraining.

Algorithm 1 is a heuristic method that calculates the optimal unit clustering subject to a user-defined training time constraint T_{th}. The algorithm takes as inputs the optimal look-back window N determined by the statistical method described earlier, the number of units Q, the total number of appliances M, and the training time threshold T_{th}. It begins by initializing the number of clusters and iteratively partitions the dataset into an increasing number of clusters. For each cluster, it trains a separate model using the appliances within that cluster and records the training time. The outputs of all models are then aggregated. The algorithm evaluates the aggregated output if the training time is within the specified threshold. The process continues until the algorithm finds the optimal cluster size that meets the training time constraint. The output of the algorithm is this optimal cluster size, ensuring efficient training within the given time constraints.

Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.

B. Comparisons With Alternative Forecasting Models

To evaluate the impact of the proposed model we compared it against various existing models in the literature.

- We compared against an implementation of a $BiLT_E$ model where the first component is a bidirectional LSTM consisting of K layers, each of L cells, which feeds an encoder-only Transformer model to output the final prediction. The second component is structurally identical to the one of the proposed LT_E.
- We compared against an alternative hybrid architecture named T_EL, which, like LT_E, combines Transformer and LSTM modules, but in reverse order. We implemented the T_EL model, where the input sequence is initially directed to a Transformer encoder as in [32], composed of P layers each containing R attention heads, which transforms the sequence into vector embeddings. These embeddings are then fed into the LSTM module to generate the final output predictions. The LSTM component of the TL architecture consists of K layers, with each layer having L cells and its look-back value N is optimized. It is responsible for producing the predicted power load for subsequent time points. The output of the Transformer encoder, a vector of size T, is inputted directly into this LSTM module.
- We compared against a hybrid encoder-only Transformer to biLSTM T_EBiL model similar to [36] and also against a full-body Transformer $TBiL$, both utilizing bidirectional LSTM components.
- We compared the LT_E architecture approach against the optimized LSTM models in [14].
- We also compared our results with enhanced LSTM structures such as the bidirectional LSTM (BiLSTM) model in [16], which we label as BiL. We employed their multi-learning strategy adjusted to our task, by utilizing three multivariate BiLSTM components, each dedicated to a specific load category: two for the dominant load categories and a third for all other loads. These components feed into a shared fully connected neural network layer consisting of 10 neurons, which collectively output the total load prediction.
- In addition, we conducted a comparison between the proposed LT_E models and two standalone Transformer models. The first model encompasses the complete encoder-decoder architecture, as outlined in [32], and is denoted as T. In this model, we used one decoder since we forecast a single load. The second model comprises an encoder module only, noted as T_E. The construction of T_E also followed the approach detailed in [32].
- Finally, we compared against the GNN approach in [13].

C. Forecasting a Sequence of Loads

To accommodate consecutive load sequence prediction, we augment the Transformer part of the hybrid LT_E model by incorporating a decoder module. The encoder-decoder part of the modified Transformer module of the LT_E model for this task, called LT, is depicted in Figure 5, and is similar to the structure in [17]. The decoder is responsible for generating

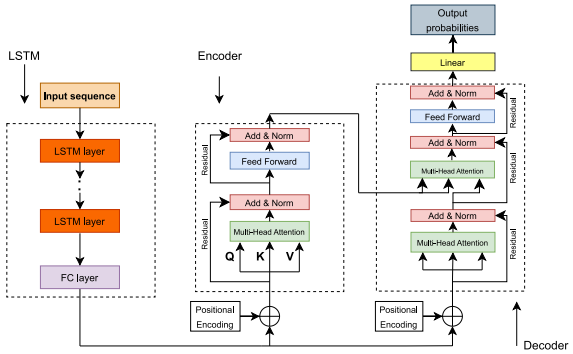


Fig. 5. The proposed hybrid LT model.

predictions for a sequence of future time instances. Each decoder layer receives the embeddings of the input time series from the encoder and predicts the load for a specific future time step. The encoder structure of the LT model is identical to the LT_E model as described earlier, except for omitting the last two layers. Hence, the embeddings produced by the encoder are directed to the decoder module for the final load sequence prediction.

The added decoder module is structurally similar to the encoder, but it includes additional components such as an intermediate Multi-Head Attention layer fed by the encoder's output, as described in II-B. The output of the decoder is processed through a Linear layer to compute the continuous-valued final output of the LT architecture, suitable for regression tasks like power load forecasting.

For this particular task, the MAE is defined as:

$$MAE = \frac{1}{F}\sum_{j=1}^{F}\sum_{i=1}^{H}|\hat{y}_{ij} - y_{ij}| \tag{5}$$

where F is the total number of prediction steps in the look-forward window, H is the number of individual prediction instances within each prediction step, $\hat{y}_{ij}$ represents the predicted value for the ith instance in the jth prediction step, and y_{ij} represents the actual observed value for the ith instance in the jth prediction step. The PMAE is defined as in Eq.4.

We also implemented an alternative hybrid model, denoted as TL, in addition to the single instance prediction with the hybrid T_EL model. The TL model integrates a full-body encoder-decoder Transformer, akin to the structure described in [17], with an LSTM module for the final output prediction across a multi-step window. This Transformer, structurally similar to that in the LT model, generates embedding vectors. The key distinction lies in the output of the decoder: it produces a vector of size T, which is subsequently fed into the LSTM module for processing. This design choice allows the model to effectively handle sequence predictions.

IV. Experimental Results

This study assumes the availability of historical data on household electricity consumption via their smart grids. In all our experiments, we have made use of publicly accessible data on historical electric load consumption provided by the National Renewable Energy Laboratory (NREL) as

Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.

outlined in [42]. This dataset includes time series data on energy consumption for various building types (residential and commercial) and end uses, considering different climate regions in the United States, organized by states. Specifically, we have obtained data for the power consumption of 650 residents in California, Illinois, and Texas for the year 2018 in [42]. This dataset provides detailed observations of electric load consumption, disaggregated by individual appliances, and measured in 15-minute intervals. For all the experiments, the respective data was split into a training set and a validation set, which is unseen during training. The size of the training set was by 33.3% larger than the size of the validation set. All our experiments were conducted on a workstation with the following specifications: an 8-core Intel(R) Core(TM) i7-9700 processor running at 3.00GHz, 32GB of RAM, and a GeForce RTX 2090 graphics processing unit. This workstation operated on the Linux Ubuntu 20.04 operating system, and we employed Pytorch 1.4 with CUDA 10.2. It's crucial to emphasize that the primary goal of this study is to enhance the performance of the proposed hybrid LT_E model for power forecasting, regardless of the specific input appliances used.

We conducted an evaluation of the proposed LT_E architecture, which was optimized using the statistical approach for determining the optimal look-back parameter, as outlined in Section III-A. This method was also applied to all the forecasting models that are compared in this experiment. The LT_E models reported in this section were designed with $K = 2$ LSTM layers, $L = 32$ LSTM cells per layer, $T = 256$ dimension of the embedding vectors of the LSTM module, $P = 6$ stacked encoder layers, and $R = 4$ attention heads. The query (Q), key (K), and value (V) matrices in the multi-head attention mechanism each had a dimension of 64 per head. The feed-forward network in each Transformer layer of the encoder (and the decoder of the LT model) expands the embedding size to 1024, which is 4 times the input dimension, followed by a projection back to the original 256-dimensional space before predicting the final regression output. These parameters, as well as the maximum look-back window, $D = 48h$, were selected after a thorough examination of the residential data pool and the performance of the trained LT_E models. For each model comprising LSTM components, including ours, we implemented both unidirectional and bidirectional versions. Since bidirectional LSTMs are more computationally expensive, we report the bidirectional versions in Table I only when they outperform their respective unidirectional counterparts. For example, the bidirectional model $BiLT_E$ did not improve performance over the unidirectional LT_E, and is therefore excluded from the reported results. Similarly, $BiLT$ did not surpass LT, and is also omitted. In contrast, BiL demonstrated better performance than L, so it is included.

In our research, we empirically determined the optimal look-back window for each implemented model using data obtained from a pool of 650 households. We created 150 distinct clusters of houses, each varying in size, with an equal representation of 50 clusters from each state, achieved through random selection from the overall data pool. The range of cluster sizes was from 1 to 100 houses. We then trained each forecasting model across all these clusters, assessing the average optimal input sequence length for these varied setups. The performance of each model was evaluated both on a state-wise basis and collectively, focusing on achieving decent accuracy across the majority of data while avoiding extensive individual optimizations.

TABLE I
PMAE VALUES FOR THE PROPOSED LT_E AND OTHER FORECASTING MODELS

model	CA $\times 10^{-5}$	IL $\times 10^{-5}$	TX $\times 10^{-5}$	overall $\times 10^{-5}$	std $\times 10^{-5}$
LT_E(proposed)	**1.24**	**1.22**	**1.29**	**1.25**	**1.58**
LT	1.24	1.22	1.30	1.25	1.58
T [32]	1.78	1.81	1.84	1.81	1.93
BiL [16]	1.96	1.96	2.01	1.98	2.03
L [14]	1.98	1.99	2.00	1.99	2.17
TL	2.00	2.00	2.01	2.00	2.33
T_EBiL [36]	1.99	2.00	2.02	2.00	2.34
T_EL	2.01	1.98	2.03	2.01	2.35
T_E [32]	2.13	2.08	2.08	2.10	2.47
GNN [13]	2.26	2.22	2.24	2.24	3.11
L [7]	2.38	2.44	2.45	2.42	3.78

Table I presents a comparative analysis of the performance of various models. They include the models listed in Section III-B, using the Percentage MAE metric as defined in Eq. (4), for each state separately, and for all three states combined. The models are sorted by their overall PMAE performance. Columns 2, 3, and 4 report the per-state PMAE $\times 10^{-5}$ performance of each model in California, Illinois, and Texas, and the fifth column reports the combined error in all three states. The standard deviation of each distribution of the prediction errors across the entire dataset is reported in the last column. The LT_E and the LT models consistently outperformed the other models, indicating their superior performance. Their PMAE overall error was 1.25×10^{-5} and their standard deviation was 1.58×10^{-5} as shown in columns 5 and 6. However, both the training and the inference time of the LT are on average increased by 28.7% than the ones of the LT_E. Among the other models, the T ranked third, BiL in [16] ranked fourth, the L in [14] ranked fifth, and the hybrid TL ranked sixth. Their PMAE errors and their standard deviations were significantly higher than those of the proposed LT_E and LT. The PMAE errors of the remaining models exceed the value of 2×10^{-5} and their standard deviations are even higher, and we do not consider these models in our further analysis. The standalone BiLSTM in [16] demonstrated only marginal improvements in accuracy when compared to the standalone standard LSTM. In general, the enhanced LSTM structures when integrated as the initial component of the hybrid LT architecture, failed to exhibit any significant impact on the results, regardless of whether they were paired with the LT or LT_E variation.

In Figure 6, the histogram of sub-figure (a) depicts the percentage frequency distribution of prediction errors (measured in units of PMAE $\times 10^{-5}$) for the LT_E model, covering a comprehensive sample of 150 clusters of variously sized clusters of houses. To facilitate better visualization, we employed a logarithmic scale on the y-axis. Notably, the data points predominantly clustered between 0 and 2.5, with a peak at

Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.

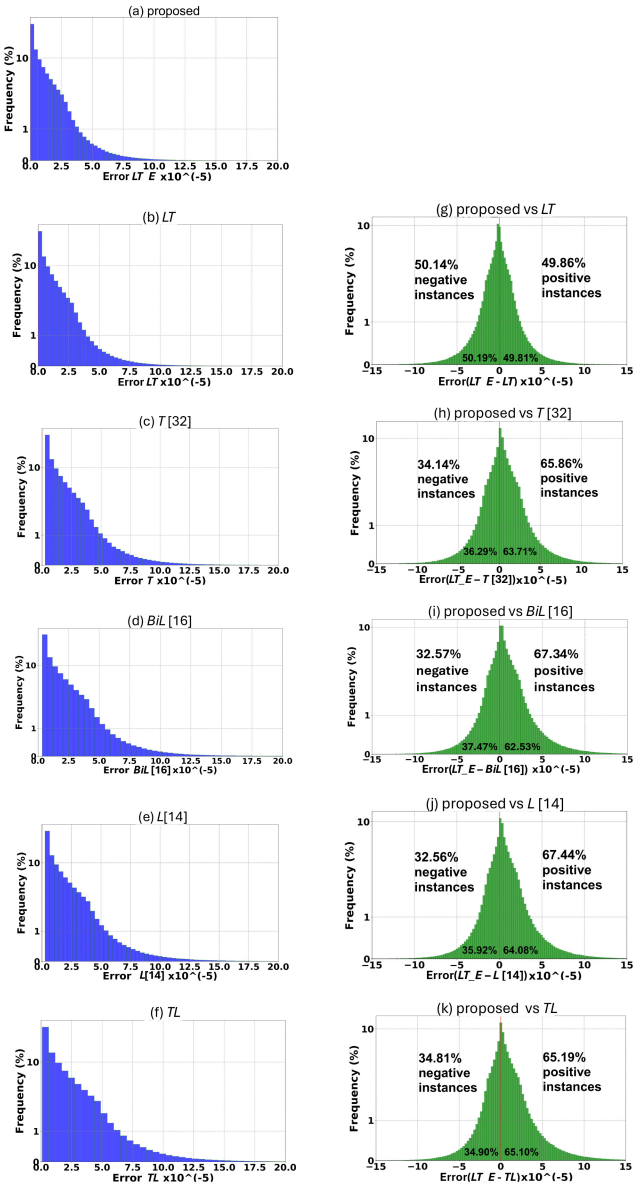


Fig. 6. Frequency (%) distributions of the PMAE error for various models, as shown in (a)-(f). Comparison of LT_E to other models as shown in (g)-(k).

zero along the x-axis. The distribution's mean was 1.25, and it had a standard deviation of 1.58. As this distribution included all cases and experiments, it signified a vast and varied dataset, highlighting the LT_E model's extensive applicability and reliability across diverse conditions. Adjacent to this, the sub-figures (b)-(f) in Figure 6 illustrated the percentage frequency distributions of prediction errors in PMAE $\times 10^{-5}$ for models ranking below LT_E. Notably, the LT model's distribution closely mirrored that of the LT_E, corroborated by identical means and standard deviations, and both approximated a half-bell normal distribution shape. Conversely, the T model's error distribution was skewed rightward, with a mean of 1.81 and a standard deviation of 1.93, predominantly ranging between 0 and 4. The distributions for the *BiL* [16], the L, and TL models exhibited more pronounced rightward tails, with means of 1.99, 2.00, and 2.01, and standard deviations of 2.03, 2.33, and 2.35, respectively, indicating a wider dispersion of errors. These variations in distribution shapes and statistics across the models provided insights into their relative performance and error characteristics.

The sub-figures (g)-(k) of Figure 6 present the error differential between the LT_E model and the five subsequent top-ranking models, quantified in units of PMAE $\times 10^{-5}$. These plots show the proportion of instances where the LT_E model's error was lower (positive cases) versus when it was higher (negative cases) than that of the compared model. This differential serves as an indicator of the frequency with which the LT_E model's predictions more closely align with the ground truth relative to the other models. Additionally, we quantified the contribution of the differential error by calculating the percentage of the area under the curve to the left and right of zero, which assesses the product of the prediction error times the corresponding frequency. The area under the curve represents a comprehensive measure of performance, quantifying not just the frequency with which LT_E outperforms the comparison model, but also incorporating the magnitude of the difference in their prediction errors.

In the comparison depicted in subplot (g), the LT_E and LT models exhibited a nearly balanced distribution of positive and negative instances, at 50.14% and 49.86%, respectively. The area under the curve metrics were similarly balanced at 50.19% and 49.81%, suggesting comparable performance between the two models. The comparison between the Transformer T model in [32] and LT_E, in Fig.6(h), revealed a marked difference, with 65.86% positive instances and 63.71% in the positive area under the curve favoring the LT_E model. The comparison between LT_E and the *BiL* in [16] in subplot (i) showed a 67.43% positive instances and a 62.53% positive area under the curve. Similarly, the comparison involving LT_E and the L model (referenced in an optimal LSTM study) depicted in the right subplot (j) of Fig.6 indicated that 67.44% of instances and 64.08% of the area under the curve were positive, again favoring the LT_E model. Finally, the comparison between LT_E and the TL model in Fig.6(k) demonstrated a notable performance differential, with 65.19% of instances and 65.10% of the area under the curve being positive in favor of the LT_E model. These comparisons collectively underscore the superior performance of the LT_E model against its counterparts across various metrics.

Training duration is a critical factor in the evaluation of each model's practicality. Notably, models incorporating an encoder-decoder Transformer architecture require approximately 30% more training time than their encoder-only counterparts. In our evaluations, the L model, as cited in [14], was the fastest, registering an average training duration of 1484.03 seconds. The LT_E model required 34.3% more time than the L model, averaging 1993.05 seconds. However, this increased training time is balanced by its enhanced accuracy. The average training time of the T model [32] was 1952.06 seconds, 2.1% less than the LT_E. In contrast, the LT model's training time was 28.7% longer than that of LT_E, tallying at 2565.06 seconds. The average training time of the *BiL* model in [16] was 18.3% greater than the L, due to its structural complexity. Given that the performance metrics of

Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.

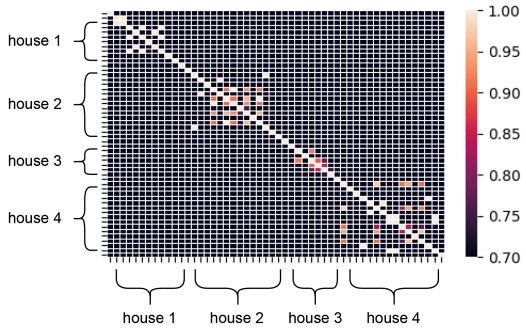


Fig. 7. Square matrix of the pair-wise correlations between the appliances of a cluster of 4 houses.

the LT and LT_E models are nearly identical, the LT_E model is preferable due to its relatively shorter training time. This preference highlights the significance of training efficiency, which is vital for practical applications where models may need to be retrained, such as in scenarios involving the change in ownership of a residence.

We assessed the performance of Algorithm 1 on the data pool, which optimizes the clustering of a given number of appliances given a time constraint on the training time of the LT_E model. Firstly, to validate the assumption that intra-house appliances exhibit patterns more similar than inter-house appliances, we conducted an analysis of pairwise Pearson correlation coefficients between appliances within clusters of houses. The Pearson correlation coefficient [43], is a statistical measure that quantifies the strength and direction of a linear relationship between two continuous variables. Its absolute values range from 0 to 1, where 1 indicates a perfect positive linear relationship, and 0 indicates no linear relationship. Figure 7 displays the correlations between appliances in a cluster of 4 units. Each element in the symmetric square matrix represents the Pearson correlation coefficient between a pair of appliances within the group. The axes denote the appliances of the four houses, and the main diagonal represents correlations between identical appliances, yielding a value of 1. Notably, strong correlations (indicated by red, orange, and white colors) are observed exclusively between appliances belonging to the same household. The results unequivocally demonstrate that time-series appliances within the same house exhibit strong positive correlations. Conversely, appliances from different houses show negligible correlation with one another.

The experimental setup involved random selection of $Q = 100$ houses from the data pool. A number of units per predictor equal to $n = 100$ means that all the load series of the examined cluster were used as inputs in a single LT_E model, that outputs the total power consumption of the cluster of 100 units. When $n = 1$, 100 LT_E models were independently trained, each on a single resident that outputs its total power consumption, and the outputs of the 100 models were aggregated to calculate the total power consumption of the cluster of 100 units. The experiments were repeated 20 times and the results of the averaged errors (measured in units of PMAE $\times 10^{-5}$) are presented in the accompanying Table II. The observation is that, as the number of units per predictor decreases the averaged MAE error increases, which is in agreement with the reported results in [14]. For a single LT_E model predictor accommodating all the load series of a cluster of 100 residents, we got the minimum average error value of 1.26×10^{-5} PMAE.

TABLE II
THE LT_E AVERAGED ERROR (PMAE) AND THE TRAINING TIME (S) FOR DIFFERENT CLUSTERS OF APPLIANCES

# of predictors	units per predictor	PMAE $\times 10^{-5}$	tr. time (s)
1	100	1.26	1993.05
2	50	1.33	996.63
10	10	1.39	198.82
100	1	1.43	17.46

Table II presents an overview of the average training times for the LT_E models across various grouping scenarios, illustrating a linear relationship between the training time and the number of input units or variables. It's important to note that, except for when $n = 100$, where all input appliances were used in a single model, the reported training times refer to the training time for a single group. This assumes that all groups can be trained simultaneously as separate and independent processes.

To replicate real-world scenarios with limited training time, we assessed the effectiveness of Algorithm1, in minimizing prediction error while adhering to time constraints for the cases of 200 seconds and 1000 seconds. In the case of the 200-second threshold, Algorithm 1 determined that an optimal grouping of 10 units per predictor was ideal. This configuration allowed the LT_E model to achieve a PMAE of 1.39×10^{-5}. For the 1000-second threshold, the optimal group size per predictor was 50 units. This setup led to the LT_E model achieving a PMAE of 1.33×10^{-5}.

We also explored the extension of load prediction from the next time instance to the sequence forecasting, modifying the proposed hybrid LSTM-Transformer model, to predict multiple future time instances rather than just the very next time instance. The architecture of the modified model, denoted by LT, is described in Section III-A and is shown in Figure 5.

Additionally, we report the experiments conducted on the sequence forecasting task utilizing the proposed encoder-only LT_E model. Unlike the next time instance prediction for which the LT_E model was initially designed, predicting a sequence of future time instances presents a challenge. The LT_E hybrid model, in its original design in Section III-A, is not inherently structured to predict a sequence of future time instances directly. To address this limitation, we devised an indirect prediction approach. In this method, the output of the LT_E model for a specific time instance $t + 1$ was utilized as input to predict the load for the subsequent time instance $t + 2$. This process was repeated iteratively until the desired sequence length was achieved. It's important to acknowledge that utilizing such an indirect approach introduces a potential challenge regarding prediction accuracy. With each iteration of prediction, the error from the previous step is propagated and compounded into the calculation of the subsequent time instance's load. Consequently, this cumulative error propagation has the potential to increase the overall

Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON BETWEEN LT_E, LT, AND TL MODELS USING THE PMAE IN EQ. (4)

seq length	model	CA $\times 10^{-5}$	IL $\times 10^{-5}$	TX $\times 10^{-5}$	overall $\times 10^{-5}$
4	LT	**1.27**	**1.28**	**1.27**	**1.27**
	LT_E	1.30	1.29	1.30	1.30
	TL	2.00	2.14	2.08	2.07
8	LT	**1.29**	**1.28**	**1.30**	**1.29**
	LT_E	1.38	1.37	1.38	1.38
	TL	2.10	2.18	2.10	2.13
12	LT	**1.32**	**1.31**	**1.33**	**1.32**
	LT_E	1.52	1.52	1.51	1.52
	TL	2.31	2.30	2.37	2.32

prediction error as the sequence progresses further into the future.

We evaluated the performance of the two proposed hybrid architectures, the *LT* model (an encoder-decoder structure) and the modified encoder-only LT_E model, to verify their performance in sequence-to-sequence load forecasting tasks. We additionally compared against the *TL* where the LSTM component outputs the sequential prediction. The tasks considered predicting electrical load for future intervals set at 4, 8, and 12 instances ahead, each instance representing 15 minutes. We carried out each experiment across a sample space encompassing 150 clusters of houses, with each cluster drawn evenly from three U.S. states, thus totaling 50 clusters per state. We documented the PMAE scores (measured in units of PMAE $\times 10^{-5}$) for each state and the average overall PMAE for both models in Table III. Results demonstrated that the *LT* model consistently outperformed both the LT_E and the *TL* models across all sequence lengths and geographic regions.

More precisely, at a 4-instance sequence length, the *LT* model yielded an overall PMAE of 1.27, compared to 1.30 by the LT_E model and 2.07 by the *TL* model. At an 8-instance sequence length, the *LT* recorded a PMAE of 1.29, while the LT_E registered 1.38 and the *TL* registered 2.13. At the 12-instance length, the *LT* model achieved a PMAE of 1.32, significantly lower than the 1.52 observed for the LT_E model, and the 2.32 of the *TL* model.

The less favorable performance of the LT_E model is attributed to its indirect prediction methodology, wherein each output prediction feeds as the input for the next time instance, leading to error propagation and amplified inaccuracies over longer sequence lengths. In contrast, the *LT* model, with its encoder-decoder framework, does not suffer from the same cumulative error, rendering it more effective for extensive sequence predictions and demonstrating its robustness in complex forecasting scenarios. Similar to the single instance prediction results, the *TL* model exhibits significantly lower performance compared to the LT_E and *LT* models, underscoring the superiority of Transformer models over LSTM in handling the prediction aspect of the process.

V. CONCLUSION

In this paper, we presented a novel hybrid LSTM-Transformer architecture tailored for the task of power load forecasting. Two variants of the model were proposed for the next time load instance predictions and sequential load predictions, respectively. To improve performance, we delved into optimization strategies such as adjusting the look-back parameter to reduce prediction errors and selectively incorporating load time series clusters, aiming for enhanced forecasting accuracy within the training time limitations often faced in real-world deployments. Both variants of the proposed model were systematically examined. We demonstrated their effectiveness against several existing load forecasting models.

REFERENCES

[1] S. Karthika, V. Margaret, and K. Balaraman, "Hybrid short term load forecasting using ARIMA-SVM," in *Proc. Innov. Power Adv. Comput. Technol. (i-PACT)*, 2017, pp. 1–7.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] B. Stephen, X. Tang, P. R. Harvey, S. Galloway, and K. I. Jennett, "Incorporating practice theory in sub-profile models for short term aggregated residential load forecasting," *IEEE Trans. Smart Grid*, vol. 8, no. 4, pp. 1591–1598, Jul. 2017.

[4] F. L. Quilumba, W.-J. Lee, H. Huang, D. Y. Wang, and R. L. Szabados, "Using smart meter data to improve the accuracy of intraday load forecasting considering customer behavior similarities," *IEEE Trans. Smart Grid*, vol. 6, no. 2, pp. 911–918, Mar. 2015.

[5] C. Shang, J. Gao, H. Liu, and F. Liu, "Short-term load forecasting based on PSO-KFCM daily load curve clustering and CNN-LSTM model," *IEEE Access*, vol. 9, pp. 50344–50357, 2021.

[6] P. Zhang, X. Wu, X. Wang, and S. Bi, "Short-term load forecasting based on big data technologies," *CSEE J. Power Energy Syst.*, vol. 1, no. 3, pp. 59–67, Sep. 2015.

[7] W. Kong, Z. Y. Dong, D. J. Hill, F. Luo, and Y. Xu, "Short-term residential load forecasting based on resident behaviour learning," *IEEE Trans. Power Syst.*, vol. 33, no. 1, pp. 1087–1088, Jan. 2018.

[8] W. Kong, Z. Y. Dong, Y. Jia, D. J. Hill, Y. Xu, and Y. Zhang, "Short-term residential load forecasting based on LSTM recurrent neural network," *IEEE Trans. Smart Grid*, vol. 10, no. 1, pp. 841–851, Jan. 2019.

[9] R. Wang and J. Zhao, "Deep learning-based short-term load forecasting for transformers in distribution grid," *Int. J. Comput. Intell. Syst.*, vol. 14, no. 1, pp. 1–10, 2021.

[10] P. Li, S. He, P. Han, M. Zheng, M. Huang, and J. Sun, "Short-term load forecasting of smart grid based on long-short-term memory recurrent neural networks in condition of real-time electricity price," *Power Syst. Technol.*, vol. 42, no. 12, pp. 4045–4052, 2018.

[11] G. Gong, X. An, N. K. Mahato, S. Sun, S. Chen, and Y. Wen, "Research on short-term load prediction based on seq2seq model," *Energies*, vol. 12, no. 16, p. 3199, 2019.

[12] S. H. Rafi, Nahid-Al-Masood, S. R. Deeba, and E. Hossain, "A short-term load forecasting method using integrated CNN and LSTM network," *IEEE Access*, vol. 9, pp. 32436–32448, 2021.

[13] W. Lin, D. Wu, and B. Boulet, "Spatial-temporal residential short-term load forecasting via graph neural networks," *IEEE Trans. Smart Grid*, vol. 12, no. 6, pp. 5373–5384, Nov. 2021.

[14] V. Pentsos, S. Tragoudas, J. Wibbenmeyer, and N. Khdeer, "Optimizing multivariate LSTM on real-world data for power load forecasting," in *Proc. IEEE 11th Int. Conf. Smart Energy Grid Eng. (SEGE)*, 2023, pp. 126–130.

[15] Z. Huang, W. Xu, and K. Yu, "Bidirectional LSTM-CRF models for sequence tagging," 2015, *arXiv:1508.01991*.

[16] Y. Guo et al., "BiLSTM multitask learning-based combined load forecasting considering the loads coupling relationship for multienergy system," *IEEE Trans. Smart Grid*, vol. 13, no. 5, pp. 3481–3492, Sep. 2022.

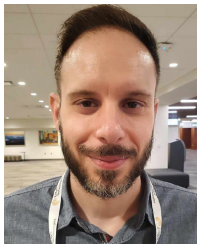
[17] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Conf. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–15.

[18] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empir. Methods Nat. Lang. Process., Syst. Demonstr.*, 2020, pp. 38–45.

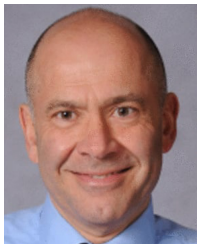
[19] H. Wang et al., "Hat: Hardware-aware transformers for efficient natural language processing," 2020, *arXiv:2005.14187*.

Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.

[20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
[21] J. Vig, "Visualizing attention in transformer-based language representation models," 2019, *arXiv:1904.02679*.
[22] G. Zhang, C. Wei, C. Jing, and Y. Wang, "Short-term electrical load forecasting based on time augmented transformer," *Int. J. Comput. Intell. Syst.*, vol. 15, no. 1, p. 67, 2022.
[23] N. D. Tuyen, N. T. Thanh, V. X. S. Huu, and G. Fujita, "A combination of novel hybrid deep learning model and quantile regression for short-term deterministic and probabilistic PV maximum power forecasting," *IET Renew. Power Gener.*, vol. 17, no. 4, pp. 794–813, 2023.
[24] F. Andayani, L. B. Theng, M. T. Tsun, and C. Chua, "Hybrid-transformer model for emotion recognition from speech audio files," *IEEE Access*, vol. 10, pp. 36018–36027, 2022.
[25] P. Ran, K. Dong, X. Liu, and J. Wang, "Short-term load forecasting based on CEEMDAN and transformer," *Electr. Power Syst. Res.*, vol. 214, Jan. 2023, Art. no. 108885.
[26] P. C. Huy, N. Q. Minh, N. D. Tien, and T. T. Q. Anh, "Short-term electricity load forecasting based on temporal fusion transformer model," *IEEE Access*, vol. 10, pp. 106296–106304, 2022.
[27] K. Wang, J. Zhang, X. Li, and Y. Zhang, "Long-term power load forecasting using LSTM-informer with ensemble learning," *Electronics*, vol. 12, no. 10, p. 2175, 2023.
[28] C. Wang, Y. Wang, Z. Ding, T. Zheng, J. Hu, and K. Zhang, "A transformer-based method of multienergy load forecasting in integrated energy system," *IEEE Trans. Smart Grid*, vol. 13, no. 4, pp. 2703–2714, Jul. 2022.
[29] A. L'Heureux, K. Grolinger, and M. A. Capretz, "Transformer-based model for electrical load forecasting," *Energies*, vol. 15, no. 14, p. 4993, 2022.
[30] J. Zhang, H. Zhang, S. Ding, and X. Zhang, "Power consumption predicting and anomaly detection based on transformer and K-means," *Front. Energy Res.*, vol. 9, Oct. 2021, Art. no. 779587.
[31] J. Gao, Y. Chen, W. Hu, and D. Zhang, "An adaptive deep-learning load forecasting framework by integrating transformer and domain knowledge," *Adv. Appl. Energy*, vol. 10, Jun. 2023, Art. no. 100142.
[32] Z. Wang, Z. Zhu, G. Xiao, B. Bai, and Y. Zhang, "A transformer-based multi-entity load forecasting method for integrated energy systems," *Front. Energy Res.*, vol. 10, Jul. 2022, Art. no. 952420.
[33] M. Jurasovic, E. Franklin, M. Negnevitsky, and P. Scott, "Day ahead load forecasting for the modern distribution network-a tasmanian case study," in *Proc. Aust. Univ. Power Eng. Conf. (AUPEC)*, 2018, pp. 1–6.
[34] T. N. Trong, H. V. X. Son, H. Do Dinh, H. Takano, and T. N. Duc, "Short-term PV power forecast using hybrid deep learning model and variational mode decomposition," *Energy Rep.*, vol. 9, pp. 712–717, Oct. 2023.
[35] J. Roberts, "On the computational power of decoder-only transformer language models," 2023, *arXiv:2305.17026*.
[36] Q. Yan, Z. Lu, H. Liu, X. He, X. Zhang, and J. Guo, "An improved feature-time transformer encoder-bi-LSTM for short-term forecasting of user-level integrated energy loads," *Energy Build.*, vol. 297, Oct. 2023, Art. no. 113396.
[37] B. Lim, S. Ö. Arık, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *Int. J. Forecast.*, vol. 37, no. 4, pp. 1748–1764, 2021.
[38] J. Yan, Y. Xie, X. Luan, Y. Guo, Q. Gong, and S. Feng, "Caption TLSTMs: Combining transformer with LSTMs for image captioning," *Int. J. Multimedia Inf. Retr.*, vol. 11, no. 2, pp. 111–121, 2022.
[39] F. Deng, Z. Chen, Y. Liu, S. Yang, R. Hao, and L. Lyu, "A novel combination neural network based on ConvLSTM-transformer for bearing remaining useful life prediction," *Machines*, vol. 10, no. 12, p. 1226, 2022.
[40] S. Chandar, C. Sankar, E. Vorontsov, S. E. Kahou, and Y. Bengio, "Towards non-saturating recurrent units for modelling long-term dependencies," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 3280–3287.
[41] S. M. Al-Selwi, M. F. Hassan, S. J. Abdulkadir, and A. Muneer, "LSTM inefficiency in long-term dependencies regression problems," *J. Adv. Res. Appl. Sci. Eng. Technol.*, vol. 30, no. 3, pp. 16–31, 2023.
[42] E. Wilson et al., *End-use Load Profiles for the U.S. Building Stock: DOE Open Energy Data Initiative (OEDI)*, Nat. Renew. Energy Lab., Golden, CO, USA, 2021.
[43] D. Freedman, R. Pisani, and R. Purves, *Statistics (International Student Edition)*, 4th ed, R. Pisani and R. Purves, Eds., New York, NY, USA: W. W. Norton Co., 2007.



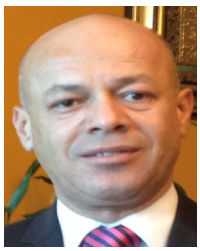
Vasileios Pentsos received the B.Sc. degree in physics from the National Kapodistrian University of Athens, Greece, and the M.Sc. degree in electrical and computer engineering from Southern Illinois University at Carbondale, where he is currently pursuing the Ph.D. degree in electrical and computer engineering, as a member of the Electronic Design and Test Automation Lab. His research areas include deep learning techniques for computer vision, AI-driven predictive modeling for embedded systems, adversarial attacks, and dynamic neural network optimization and scheduling for sustainability.



Spyros Tragoudas (Senior Member, IEEE) received the Diploma degree in computer engineering from the University of Patras, Greece, in 1986, and the M.S. and Ph.D. degrees in computer science from the University of Texas at Dallas in 1988 and 1991, respectively. He is currently a Professor and the Director of the School of Electrical, Computer, and Biomedical Engineering, Southern Illinois University at Carbondale (SIUC), where he is also the Director of the National Science Foundation Industry University Cooperative Research Center on Embedded Systems. He has held prior appointments with the Faculty of the Electrical and Computer Engineering Department, University of Arizona and the Faculty of the Computer Science Department, SIUC. His research has been funded by federal agencies and industry. He has authored or co-authored more than 300 papers in peer-reviewed journals and conference proceedings. His current research interests include VLSI design/test automation and embedded systems.



Jason Wibbenmeyer received the Bachelor of Science degree in electrical engineering from the Missouri University of Science and Technology, and the M.B.A. degree and the Master of Science degree in electrical engineering from Wright State University. He started his career with the United States Air Force, Air Force Research Laboratory, Dayton, OH, USA. He has over 25 years of experience in engineering, project management, research, and development and currently serves as the Technology Transfer Manager with Ameren, St. Louis, MO, USA. He is a Missouri licensed Professional Engineer and a certified Project Management Professional.



Nasser Khdeer received the Bachelor of Science and Master of Science degrees in electrical engineering from Southern Illinois University at Carbondale. He has over 30 years of experience in information technology, cybersecurity and project management, research, and development and currently serves as the Lead Cybersecurity Architect with the Critical Infrastructure Department, Ameren, St. Louis, MO, USA.

Authorized licensed use limited to: Indian Institute Of Technology (Banaras Hindu University) Varanasi. Downloaded on December 27,2025 at 08:12:51 UTC from IEEE Xplore. Restrictions apply.